SE3308 Algorithm Design I — Assignment 3

Exercises 4.11, 4.12, 4.16, 5.4, 5.6, and 5.10

Xilang Chen 524031910668

Write $n = |V|$ and $m = |E|$. Graphs use adjacency lists and, as usual, have no parallel edges, so $m = O(n^2)$. Undirected graphs have no self-loops. Edge-weight arithmetic and comparisons take constant time. An unreachable distance is $+\infty$; log means $\log_2$.

Exercise 4.11

Algorithm. For each vertex s, run Dijkstra's algorithm from s, using an unsorted array for the priority queue. For every incoming edge (u, s), consider the shortest path from s to u followed by (u, s). Return

$$\boxed{L = \min_{(u,v)\in E} \big(\mathrm{dist}_G(v, u) + \ell(u, v)\big).}$$

If every candidate is infinite, report that G is acyclic.

Algorithm 1 Shortest directed cycle

```
1: function ShortestCycle(G, ℓ)
2:     L ← +∞
3:     for each s ∈ V do
4:         d ← ArrayDijkstra(G, ℓ, s)
5:         for each incoming edge (u, s) do
6:             L ← min{L, d[u] + ℓ(u, s)}
7:         end for
8:     end for
9:     if L = +∞ then
10:        return acyclic
11:    end if
12:    return L
13: end function
```

Correctness. Positive edge lengths imply that a shortest path between distinct vertices can be taken to be simple: removing a repeated-vertex segment strictly shortens a walk. Thus, whenever $\mathrm{dist}_G(v, u)$ is finite, a shortest v–u path followed by (u, v) is a directed simple cycle. The path cannot already use (u, v), since that would revisit its starting vertex v. If self-loops are permitted, (v, v) itself is a cycle and the same formula uses $\mathrm{dist}_G(v, v) = 0$.

Let OPT be the length of a shortest cycle, when one exists. Every finite candidate is the length of a cycle, so $L \geq \mathrm{OPT}$. Conversely, choose an edge (u, v) of a shortest cycle C. Removing that edge leaves a v–u path, and hence

$$L \leq \mathrm{dist}_G(v, u) + \ell(u, v) \leq \ell(C) = \mathrm{OPT}.$$

Therefore $L = \mathrm{OPT}$. A finite candidate exists if and only if a cycle exists, which also proves the acyclicity report correct.

Running time. Array-based Dijkstra performs at most n linear minimum searches and scans every edge once: $O(n^2 + m) = O(n^2)$ per source. Build incoming adjacency lists once in $O(n + m)$ time; their scans across all sources total $O(m)$. The total is

$$O\big(n(n^2 + m) + n + m\big) = \boxed{O(n^3)}.$$

Only one source's distance array is needed at a time.

Exercise 4.12

Let the specified edge be $e = \{a, b\}$, and let $G' = G - e$.

Algorithm. Run array-based Dijkstra from a in G', ignoring e in both endpoints' adjacency lists. If b is unreachable, report that no cycle contains e (equivalently, return $+\infty$). Otherwise return

$$\boxed{\ell(e) + \mathrm{dist}_{G-e}(a, b).}$$

Algorithm 2 Shortest cycle through a specified undirected edge

```
1: function ShortestCycleThroughEdge(G, ℓ, e = {a, b})
2:     d ← ArrayDijkstra(G − e, ℓ, a)
3:     if d[b] = +∞ then
4:         return no cycle contains e
5:     end if
6:     return ℓ(e) + d[b]
7: end function
```

Correctness. Removing e from any simple cycle containing e leaves an a–b path in $G - e$. Consequently every such cycle has length at least $\ell(e) + \mathrm{dist}_{G-e}(a, b)$.

Conversely, if b is reachable, take a shortest a–b path P in $G - e$. By positivity, P is simple. It avoids e, so $P \cup \{e\}$ is a simple cycle of exactly the returned length. In a simple undirected graph, P has at least two edges. If b is unreachable, the first direction shows that no cycle through e can exist.

Running time. One array-based Dijkstra run costs $O(n^2 + m) = \boxed{O(n^2)}$. Skipping e requires only constant extra work per adjacency entry; there is no need to copy the graph.

Why remove e? In the original graph, a shortest a–b path could be e itself. Following it and then traversing e backwards is not a simple cycle.

Exercise 4.16

All arrays are indexed from 1, and complete trees are stored in level order, left to right. We call the final subpart **(d)**; it is printed as a second **(a)** in the exercise.

(a) Binary-tree indices. The root is at index 1. The two child slots of each successive parent occupy the pairs

$$(2,3),\ (4,5),\ (6,7),\ \ldots.$$

Thus the children of index j are $2j$ and $2j+1$, retaining only indices at most n. Conversely, every non-root index $j \geq 2$ is either $2p$ or $2p+1$ for its parent p, so

$$\boxed{\text{parent}(j) = \lfloor j/2 \rfloor, \qquad j \geq 2.}$$

The root has no parent.

(b) d-ary-tree indices. Assume $d \geq 2$. The root's child slots start at index 2. The preceding $j-1$ parents account for $d(j-1)$ child slots, so the r-th child of parent j has index

$$\boxed{\text{child}_r(j) = d(j-1) + r + 1, \qquad 1 \leq r \leq d.}$$

Discard indices exceeding n. A non-root index q belongs to the block of its parent p exactly when

$$d(p-1) + 2 \leq q \leq d(p-1) + d + 1.$$

Therefore

$$\boxed{\text{parent}(q) = \left\lfloor \frac{q-2}{d} \right\rfloor + 1 = \left\lceil \frac{q-1}{d} \right\rceil, \qquad q \geq 2.}$$

Putting $d = 2$ recovers part (a).

(c) Linear-time makeheap and a worst-case input.

Why bottom-up heap construction works. When SIFTDOWN is called at position i, its child subtrees are already min-heaps: every child has index greater than i and was processed earlier. Moving the smaller child up until the saved item fits restores the heap property throughout the subtree rooted at i. Induction over decreasing i proves that the final array is a min-heap.

Running time. The height H_i of the subtree rooted at i is

$$H_i = \lfloor \log(n/i) \rfloor.$$

Indeed, the leftmost descendant r levels below i has index $2^r i$, so such a level exists exactly when $2^r i \leq n$. The call at i costs $O(1 + H_i)$. Using the individual heights avoids the loose upper bound $O(n \log n)$. Counting the total levels gives

$$\begin{aligned} \sum_{i=1}^{n} H_i &= \sum_{r=1}^{\lfloor \log n \rfloor} \left| \{ i : 2^r i \leq n \} \right| \\ &= \sum_{r=1}^{\lfloor \log n \rfloor} \left\lfloor \frac{n}{2^r} \right\rfloor \leq n \sum_{r=1}^{\infty} 2^{-r} = n. \end{aligned}$$

Thus filling the array and all calls together take $O(n + \sum_{i=1}^{n} H_i) = \boxed{O(n)}$. This also justifies the hint: since $\log(n/i) < H_i + 1$, we have $\sum_{i=1}^{n} \log(n/i) < 2n$. The additive $O(n)$ accounts for initialization and constant work at leaves. Initialization alone gives an $\Omega(n)$ lower bound.

Worst-case input family. For a perfect binary tree with $n = 2^{h+1} - 1$ nodes, put distinct keys in strictly decreasing array order:

$$h[1..n] = [n, n-1, \ldots, 1].$$

At the call for index i, the saved key is larger than every key in its descendant subtrees. Earlier calls have only rearranged keys within those subtrees. It therefore moves down to a leaf; because the tree is perfect, every leaf in that subtree is H_i levels below i. Every call attains its maximum possible downward distance. The number of downward moves is

$$\sum_i H_i = \sum_{r=1}^{h} (2^{h+1-r} - 1) = 2^{h+1} - h - 2 = n - h - 1.$$

This is a worst-case family with $\Theta(n)$ work, matching the upper bound. For an incomplete last level, decreasing order need not maximize each individual descent, but the worst-case asymptotic bound remains $\Theta(n)$.

(d) Adapting the pseudocode to a d-ary heap.

Use the parent and child formulas from (b). In particular, replace every parent computation in BUBBLEUP by

$$p \leftarrow \lfloor \frac{i-2}{d} \rfloor + 1 \qquad (i > 1).$$

The root case must be checked before accessing its parent. Replace MINCHILD by the following procedure.

Algorithm 3 Smallest child in a d-ary heap

```
1: function MinChild(h, i, d)
2:     a ← d(i − 1) + 2
3:     if a > |h| then
4:         return 0
5:     end if
6:     b ← min{|h|, a + d − 1}
7:     return an index in [a, b] minimizing key(h[j])
8: end function
```

The control flow of SIFTDOWN is unchanged; it now selects among up to d children. The same INSERT, DECREASEKEY, and bottom-up MAKEHEAP schemes apply. The latter may skip leaves and start at $\lfloor (n-2)/d \rfloor + 1$ when $n \geq 2$.

Maintain the inverse map $h^{-1}(x) = \text{pos}[x]$: whenever an item x is placed in slot j, also set $\text{pos}[x] \leftarrow j$. This preserves the constant-time lookup required by DECREASEKEY.

Two details also apply to the binary version:

- Figure 4.16 prints $\lceil i/2 \rceil$ in BUBBLEUP; for a 1-indexed binary heap the correct parent is $\lfloor i/2 \rfloor$.
- DELETEMIN must actually remove the last slot. Save the root and last item, reduce the logical heap size by one, and sift the saved last item from the root only if the new heap is nonempty. Remove the deleted item's inverse-map entry. Array capacity is distinct from logical heap size.

Exercise 5.4

Claim. An undirected graph with n vertices and k connected components has at least $n - k$ edges.

Proof. Start with the same n vertices and no edges, hence n components. Insert the graph's m edges one at a time. Each inserted edge either joins two components and reduces their number by one, or has both endpoints in the same component and leaves their number unchanged. After m insertions there are therefore at least $n - m$ components. The final number is k, so

$$k \geq n - m, \qquad \text{equivalently} \qquad \boxed{m \geq n - k}. \qquad \square$$

The bound is tight: a forest with k components has exactly $n - k$ edges. Equivalently, if component i has n_i vertices, its spanning tree contributes $n_i - 1$ edges, and $\sum_{i=1}^{k}(n_i - 1) = n - k$.

Exercise 5.6

We assume G is connected, as is necessary for a spanning tree to exist. For a disconnected graph, the corresponding conclusion is uniqueness of the minimum spanning forest.

Proof by exchange. Suppose two distinct MSTs T and T' exist. Let e be the lightest edge in their symmetric difference

$$E(T) \triangle E(T') = \big(E(T) \setminus E(T')\big) \cup \big(E(T') \setminus E(T)\big).$$

Interchange the names of the trees if necessary so that $e \in E(T) \setminus E(T')$.

Adding e to T' creates a unique cycle C. Some edge f of C lies in $E(T') \setminus E(T)$: otherwise every edge of C, including e, would belong to T, contradicting that T is a tree. Both e and f are in the symmetric difference. Since e is its lightest edge and all edge weights are distinct,

$$w(e) < w(f).$$

Deleting f from $T' + e$ breaks the unique cycle, leaving a spanning tree T'' with

$$w(T'') = w(T') + w(e) - w(f) < w(T').$$

This contradicts the minimality of T'. Hence $\boxed{\text{the MST is unique.}}$ $\square$

Exercise 5.10

Write $H = (V_H, E_H)$ with the weights inherited from G, and let

$$F = E(T) \cap E_H.$$

We must prove that a *single* MST of H contains every edge of F.

Since H is connected, it has an MST. Among all its MSTs choose one, say S, maximizing $|E(S) \cap F|$. Suppose for contradiction that some $e = \{u, v\} \in F$ does not belong to S.

Removing e from T partitions V into two sets A and B, with $u \in A$ and $v \in B$. The u–v path in S must contain an edge f crossing this cut. As $e \notin E(S)$, we have $f \neq e$. Moreover, e is the *only* edge of T crossing the cut defined by the two components of $T - e$. Thus $f \notin E(T)$, so in particular $f \notin F$.

We claim $w(e) \leq w(f)$. Otherwise $T - e + f$ would be a spanning tree of G of weight

$$w(T) - w(e) + w(f) < w(T),$$

contradicting that T is an MST.

Now insert e into S. Its unique cycle consists of e and the u–v path in S, so it contains f. Hence

$$S' = S + e - f$$

is a spanning tree of H. Its weight satisfies

$$w(S') = w(S) + w(e) - w(f) \leq w(S).$$

Since S is already minimum, equality holds and S' is also an MST of H. But $e \in F$ and $f \notin F$, so

$$|E(S') \cap F| = |E(S) \cap F| + 1,$$

contradicting the choice of S.

Therefore $F \subseteq E(S)$, proving

$$\boxed{T \cap H \text{ is contained in some MST of } H.}$$

The proof allows equal edge weights and does not require H to be an induced subgraph. The intersection may be a disconnected forest; the claim is that it can be extended to an MST of H.